JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Chronology

All Superinterfaces:

`Comparable<Chronology>`

All Known Implementing Classes:

`AbstractChronology`, `HijrahChronology`, `IsoChronology`, `JapaneseChronology`, `MinguoChronology`, `ThaiBuddhistChronology`

```
public interface Chronology
extends Comparable<Chronology>
```

A calendar system, used to organize and identify dates.

The main date and time API is built on the ISO calendar system. The chronology operates behind the scenes to represent the general concept of a calendar system. For example, the Japanese, Minguo, Thai Buddhist and others.

Most other calendar systems also operate on the shared concepts of year, month and day, linked to the cycles of the Earth around the Sun, and the Moon around the Earth. These shared concepts are defined by `ChronoField` and are available for use by any `Chronology` implementation:

```
LocalDate isoDate = ...
ThaiBuddhistDate thaiDate = ...
int isoYear = isoDate.get(ChronoField.YEAR);
int thaiYear = thaiDate.get(ChronoField.YEAR);
```

As shown, although the date objects are in different calendar systems, represented by different `Chronology` instances, both can be queried using the same constant on `ChronoField`. For a full discussion of the implications of this, see `ChronoLocalDate`. In general, the advice is to use the known ISO-based `LocalDate`, rather than `ChronoLocalDate`.

While a `Chronology` object typically uses `ChronoField` and is based on an era, year-of-era, month-of-year, day-of-month model of a date, this is not required. A `Chronology` instance may represent a totally different kind of calendar system, such as the Mayan.

In practical terms, the `Chronology` instance also acts as a factory. The `of(String)` method allows an instance to be looked up by identifier, while the `ofLocale(Locale)` method allows lookup by locale.

The `Chronology` instance provides a set of methods to create `ChronoLocalDate` instances. The date classes are used to manipulate specific dates.

- `dateNow()`
- `dateNow(clock)`
- `dateNow(zone)`
- `date(yearProleptic, month, day)`
- `date(era, yearOfEra, month, day)`
- `dateYearDay(yearProleptic, dayOfYear)`
- `dateYearDay(era, yearOfEra, dayOfYear)`
- `date(TemporalAccessor)`

Adding New Calendars

The set of available chronologies can be extended by applications. Adding a new calendar system requires the writing of an implementation of `Chronology`, `ChronoLocalDate` and `Era`. The majority of the logic specific to the calendar system will be in the `ChronoLocalDate` implementation. The `Chronology` implementation acts as a factory.

To permit the discovery of additional chronologies, the `ServiceLoader` is used. A file must be added to the `META-INF/services` directory with the name 'java.time.chrono.Chronology' listing the implementation classes. See the ServiceLoader for more details on service loading. For lookup by id or calendarType, the system provided calendars are found first followed by application provided calendars.

Each chronology must define a chronology ID that is unique within the system. If the chronology represents a calendar system defined by the CLDR specification then the calendar type is the concatenation of the CLDR type and, if applicable, the CLDR variant.

Implementation Requirements:

This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Subclasses should be Serializable wherever possible.

Since:

1.8

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description

int	**compareTo**(Chronology other)	Compares this chronology to another chronology.
ChronoLocalDate	**date**(int prolepticYear, int month, int dayOfMonth)	Obtains a local date in this chronology from the proleptic-year, month-of-year and day-of-month fields.
default ChronoLocalDate	**date**(Era era, int yearOfEra, int month, int dayOfMonth)	Obtains a local date in this chronology from the era, year-of-era, month-of-year and day-of-month fields.
ChronoLocalDate	**date**(TemporalAccessor tempor	Obtains a local date in this chronology from another temporal object.
ChronoLocalDate	**dateEpochDay**(long epochDay)	Obtains a local date in this chronology from the epoch-day.
default ChronoLocalDate	**dateNow**()	Obtains the current local date in this chronology from the system clock in the default time-zone.
default ChronoLocalDate	**dateNow**(Clock clock)	Obtains the current local date in this chronology from the specified clock.
default ChronoLocalDate	**dateNow**(ZoneId zone)	Obtains the current local date in this chronology from the system clock in the specified time-zone.
ChronoLocalDate	**dateYearDay**(int prolepticYear, int dayOfYear)	Obtains a local date in this chronology from the proleptic-year and day-of-year fields.
default ChronoLocalDate	**dateYearDay**(Era era, int yearOfEra, int dayOfYear)	Obtains a local date in this chronology from the

		era, year-of-era and day-of-year fields.
default long	**epochSecond**(int prolepticYear, int month, int dayOfMonth, int hour, int minute, int second, ZoneOffset zoneOffset)	Gets the number of seconds from the epoch of 1970-01-01T00:00:00Z.
default long	**epochSecond**(Era era, int yearOfEra, int month, int dayOfMonth, int hour, int minute, int second, ZoneOffset zoneOffset)	Gets the number of seconds from the epoch of 1970-01-01T00:00:00Z.
boolean	**equals**(Object obj)	Checks if this chronology is equal to another chronology.
Era	**eraOf**(int eraValue)	Creates the chronology era object from the numeric value.
List<Era>	**eras**()	Gets the list of eras for the chronology.
static Chronology	**from**(TemporalAccessor tempor	Obtains an instance of Chronology from a temporal object.
static Set<Chronology>	**getAvailableChronologies**	Returns the available chronologies.
String	**getCalendarType**()	Gets the calendar type of the calendar system.
default String	**getDisplayName**(TextStyle style, Locale locale)	Gets the textual representation of this chronology.
String	**getId**()	

		Gets the ID of the chronology.
int	**hashCode**()	A hash code for this chronology.
default boolean	**isIsoBased**()	Checks if this chronology is ISO based.
boolean	**isLeapYear**(long prolepticYear)	Checks if the specified year is a leap year.
default ChronoLocalDateTime<? extends ChronoLocalDate>	**localDateTime**(TemporalAccessor tempor	Obtains a local date-time in this chronology from another temporal object.
static Chronology	**of**(String id)	Obtains an instance of Chronology from a chronology ID or calendar system type.
static Chronology	**ofLocale**(Locale locale)	Obtains an instance of Chronology from a locale.
default ChronoPeriod	**period**(int years, int months, int days)	Obtains a period for this chronology based on years, months and days.
int	**prolepticYear**(Era era, int yearOfEra)	Calculates the proleptic-year given the era and year-of-era.
ValueRange	**range**(ChronoField field)	Gets the range of valid values for the specified field.
ChronoLocalDate	**resolveDate**(Map<TemporalField, Long> fieldValues, ResolverStyle resolverSt	Resolves parsed ChronoField values into a date during parsing.
String	**toString**()	Outputs this chronology as a String.

default ChronoZonedDateTime<? extends ChronoLocalDate>	**zonedDateTime**(Instant instant, ZoneId zone)	Obtains a ChronoZonedDateTime in this chronology from an Instant.
default ChronoZonedDateTime<? extends ChronoLocalDate>	**zonedDateTime**(TemporalAccessor tempor	Obtains a ChronoZonedDateTime in this chronology from another temporal object.

Method Details

from

`static Chronology from(TemporalAccessor temporal)`

Obtains an instance of `Chronology` from a temporal object.

This obtains a chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `Chronology`.

The conversion will obtain the chronology using `TemporalQueries.chronology()`. If the specified temporal object does not have a chronology, `IsoChronology` is returned.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `Chronology::from`.

Parameters:

`temporal` - the temporal to convert, not null

Returns:

the chronology, not null

Throws:

`DateTimeException` - if unable to convert to a `Chronology`

ofLocale

`static Chronology ofLocale(Locale locale)`

Obtains an instance of `Chronology` from a locale.

This returns a `Chronology` based on the specified locale, typically returning `IsoChronology`. Other calendar systems are only returned if they are explicitly selected within the locale.

The `Locale` class provide access to a range of information useful for localizing an application. This includes the language and region, such as "en-GB" for English as used in Great Britain.

The `Locale` class also supports an extension mechanism that can be used to identify a calendar system. The mechanism is a form of key-value pairs, where the calendar system has the key "ca". For example, the locale "en-JP-u-ca-japanese" represents the English language as used in Japan with the Japanese calendar system.

This method finds the desired calendar system in a manner equivalent to passing "ca" to `Locale.getUnicodeLocaleType(String)`. If the "ca" key is not present, then `IsoChronology` is returned.

Note that the behavior of this method differs from the older `Calendar.getInstance(Locale)` method. If that method receives a locale of "th_TH" it will return `BuddhistCalendar`. By contrast, this method will return `IsoChronology`. Passing the locale "th-TH-u-ca-buddhist" into either method will result in the Thai Buddhist calendar system and is therefore the recommended approach going forward for Thai calendar system localization.

A similar, but simpler, situation occurs for the Japanese calendar system. The locale "jp_JP_JP" has previously been used to access the calendar. However, unlike the Thai locale, "ja_JP_JP" is automatically converted by `Locale` to the modern and recommended form of "ja-JP-u-ca-japanese". Thus, there is no difference in behavior between this method and `Calendar#getInstance(Locale)`.

Parameters:

`locale` - the locale to use to obtain the calendar system, not null

Returns:

the calendar system associated with the locale, not null

Throws:

`DateTimeException` - if the locale-specified calendar cannot be found

of

```
static Chronology of(String id)
```

Obtains an instance of `Chronology` from a chronology ID or calendar system type.

This returns a chronology based on either the ID or the type. The `chronology ID` uniquely identifies the chronology. The `calendar system type` is defined by the CLDR specification.

The chronology may be a system chronology or a chronology provided by the application via ServiceLoader configuration.

Since some calendars can be customized, the ID or type typically refers to the default customization. For example, the Gregorian calendar can have multiple cutover dates from the Julian, but the lookup only provides the default cutover date.

Parameters:

`id` - the chronology ID or calendar system type, not null

Returns:

the chronology with the identifier requested, not null

Throws:

`DateTimeException` - if the chronology cannot be found

getAvailableChronologies

```
static Set<Chronology> getAvailableChronologies()
```

Returns the available chronologies.

Each returned `Chronology` is available for use in the system. The set of chronologies includes the system chronologies and any chronologies provided by the application via ServiceLoader configuration.

Returns:

the independent, modifiable set of the available chronology IDs, not null

getId

```
String getId()
```

Gets the ID of the chronology.

The ID uniquely identifies the `Chronology`. It can be used to lookup the `Chronology` using `of(String)`.

Returns:

the chronology ID, not null

See Also:

`getCalendarType()`

getCalendarType

```
String getCalendarType()
```

Gets the calendar type of the calendar system.

The calendar type is an identifier defined by the CLDR and *Unicode Locale Data Markup Language (LDML)* specifications to uniquely identify a calendar. The `getCalendarType` is the concatenation of the CLDR calendar type and the variant, if applicable, is appended separated by "-". The calendar type is used to lookup the `Chronology` using `of(String)`.

Returns:

the calendar system type, null if the calendar is not defined by CLDR/LDML

See Also:

`getId()`

date

```
default ChronoLocalDate date(Era era,
                             int yearOfEra,
                             int month,
                             int dayOfMonth)
```

Obtains a local date in this chronology from the era, year-of-era, month-of-year and day-of-month fields.

Implementation Requirements:

The default implementation combines the era and year-of-era into a proleptic year before calling `date(int, int, int)`.

Parameters:

`era` - the era of the correct type for the chronology, not null

`yearOfEra` - the chronology year-of-era

`month` - the chronology month-of-year

`dayOfMonth` - the chronology day-of-month

Returns:

the local date in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date

`ClassCastException` - if the `era` is not of the correct type for the chronology

date

```
ChronoLocalDate date(int prolepticYear,
                     int month,
                     int dayOfMonth)
```

Obtains a local date in this chronology from the proleptic-year, month-of-year and day-of-month fields.

Parameters:

`prolepticYear` - the chronology proleptic-year

`month` - the chronology month-of-year

`dayOfMonth` - the chronology day-of-month

Returns:

the local date in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date

dateYearDay

```
default ChronoLocalDate dateYearDay(Era era,
                                    int yearOfEra,
                                    int dayOfYear)
```

Obtains a local date in this chronology from the era, year-of-era and day-of-year fields.

Implementation Requirements:

The default implementation combines the era and year-of-era into a proleptic year before calling `dateYearDay(int, int)`.

Parameters:

`era` - the era of the correct type for the chronology, not null

`yearOfEra` - the chronology year-of-era

`dayOfYear` - the chronology day-of-year

Returns:

the local date in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date

`ClassCastException` - if the `era` is not of the correct type for the chronology

dateYearDay

```
ChronoLocalDate dateYearDay(int prolepticYear,
                            int dayOfYear)
```

Obtains a local date in this chronology from the proleptic-year and day-of-year fields.

Parameters:

`prolepticYear` - the chronology proleptic-year

`dayOfYear` - the chronology day-of-year

Returns:

the local date in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date

dateEpochDay

`ChronoLocalDate dateEpochDay(long epochDay)`

Obtains a local date in this chronology from the epoch-day.

The definition of `EPOCH_DAY` is the same for all calendar systems, thus it can be used for conversion.

Parameters:

`epochDay` - the epoch day

Returns:

the local date in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date

dateNow

`default ChronoLocalDate dateNow()`

Obtains the current local date in this chronology from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Implementation Requirements:

The default implementation invokes `dateNow(Clock)`.

Returns:

the current local date using the system clock and default time-zone, not null

Throws:

`DateTimeException` - if unable to create the date

dateNow

`default ChronoLocalDate dateNow(ZoneId zone)`

Obtains the current local date in this chronology from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Implementation Requirements:

The default implementation invokes `dateNow(Clock)`.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current local date using the system clock, not null

Throws:

`DateTimeException` - if unable to create the date

dateNow

```
default ChronoLocalDate dateNow(Clock clock)
```

Obtains the current local date in this chronology from the specified clock.

This will query the specified clock to obtain the current date - today. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Implementation Requirements:

The default implementation invokes `date(TemporalAccessor)`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current local date, not null

Throws:

`DateTimeException` - if unable to create the date

date

```
ChronoLocalDate date(TemporalAccessor temporal)
```

Obtains a local date in this chronology from another temporal object.

This obtains a date in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDate`.

The conversion typically uses the `EPOCH_DAY` field, which is standardized across calendar systems.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::date`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local date in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date

See Also:

`ChronoLocalDate.from(TemporalAccessor)`

localDateTime

```
default ChronoLocalDateTime<? extends ChronoLocalDate> localDateTime
(TemporalAccessor temporal)
```

Obtains a local date-time in this chronology from another temporal object.

This obtains a date-time in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDateTime`.

The conversion extracts and combines the `ChronoLocalDate` and the `LocalTime` from the temporal object. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects. The result uses this chronology.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::localDateTime`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local date-time in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date-time

See Also:

ChronoLocalDateTime.from(TemporalAccessor)

zonedDateTime

```
default ChronoZonedDateTime<? extends ChronoLocalDate> zonedDateTime
(TemporalAccessor temporal)
```

Obtains a `ChronoZonedDateTime` in this chronology from another temporal object.

This obtains a zoned date-time in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoZonedDateTime`.

The conversion will first obtain a `ZoneId` from the temporal object, falling back to a `ZoneOffset` if necessary. It will then try to obtain an `Instant`, falling back to a `ChronoLocalDateTime` if necessary. The result will be either the combination of `ZoneId` or `ZoneOffset` with `Instant` or `ChronoLocalDateTime`. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects. The result uses this chronology.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::zonedDateTime`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the zoned date-time in this chronology, not null

Throws:

`DateTimeException` - if unable to create the date-time

See Also:

ChronoZonedDateTime.from(TemporalAccessor)

zonedDateTime

```
default ChronoZonedDateTime<? extends ChronoLocalDate> zonedDateTime
(Instant instant,
 ZoneId zone)
```

Obtains a `ChronoZonedDateTime` in this chronology from an `Instant`.

This obtains a zoned date-time with the same instant as that specified.

Parameters:

`instant` - the instant to create the date-time from, not null

`zone` - the time-zone, not null

Returns:

the zoned date-time, not null

Throws:

`DateTimeException` - if the result exceeds the supported range

isLeapYear

```
boolean isLeapYear(long prolepticYear)
```

Checks if the specified year is a leap year.

A leap-year is a year of a longer length than normal. The exact meaning is determined by the chronology according to the following constraints.

- a leap-year must imply a year-length longer than a non leap-year.
- a chronology that does not support the concept of a year must return false.
- the correct result must be returned for all years within the valid range of years for the chronology.

Outside the range of valid years an implementation is free to return either a best guess or false. An implementation must not throw an exception, even if the year is outside the range of valid years.

Parameters:

`prolepticYear` - the proleptic-year to check, not validated for range

Returns:

true if the year is a leap year

prolepticYear

```
int prolepticYear(Era era,
                  int yearOfEra)
```

Calculates the proleptic-year given the era and year-of-era.

This combines the era and year-of-era into the single proleptic-year field.

If the chronology makes active use of eras, such as `JapaneseChronology` then the year-of-era will be validated against the era. For other chronologies, validation is optional.

Parameters:

`era` - the era of the correct type for the chronology, not null

`yearOfEra` - the chronology year-of-era

Returns:

the proleptic-year

Throws:

`DateTimeException` - if unable to convert to a proleptic-year, such as if the year is invalid for the era

`ClassCastException` - if the `era` is not of the correct type for the chronology

eraOf

`Era eraOf(int eraValue)`

Creates the chronology era object from the numeric value.

The era is, conceptually, the largest division of the time-line. Most calendar systems have a single epoch dividing the time-line into two eras. However, some have multiple eras, such as one for the reign of each leader. The exact meaning is determined by the chronology according to the following constraints.

The era in use at 1970-01-01 must have the value 1. Later eras must have sequentially higher values. Earlier eras must have sequentially lower values. Each chronology must refer to an enum or similar singleton to provide the era values.

This method returns the singleton era of the correct type for the specified era value.

Parameters:

`eraValue` - the era value

Returns:

the calendar system era, not null

Throws:

`DateTimeException` - if unable to create the era

eras

`List<Era> eras()`

Gets the list of eras for the chronology.

Most calendar systems have an era, within which the year has meaning. If the calendar system does not support the concept of eras, an empty list must be returned.

Returns:

the list of eras for the chronology, may be immutable, not null

range

```
ValueRange range(ChronoField field)
```

Gets the range of valid values for the specified field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

This method will return a result whether or not the chronology supports the field.

Parameters:

`field` - the field to get the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

getDisplayName

```
default String getDisplayName(TextStyle style,
                              Locale locale)
```

Gets the textual representation of this chronology.

This returns the textual name used to identify the chronology, suitable for presentation to the user. The parameters control the style of the returned text and the locale.

Implementation Requirements:

The default implementation behaves as though the formatter was used to format the chronology textual name.

Parameters:

`style` - the style of the text required, not null

`locale` - the locale to use, not null

Returns:

the text value of the chronology, not null

resolveDate

```
ChronoLocalDate resolveDate(Map<TemporalField, Long> fieldValues,
                            ResolverStyle resolverStyle)
```

Resolves parsed `ChronoField` values into a date during parsing.

Most `TemporalField` implementations are resolved using the resolve method on the field. By contrast, the `ChronoField` class defines fields that only have meaning relative to the chronology. As such, `ChronoField` date fields are resolved here in the context of a specific chronology.

The default implementation, which explains typical resolve behaviour, is provided in `AbstractChronology`.

Parameters:

`fieldValues` - the map of fields to values, which can be updated, not null

`resolverStyle` - the requested type of resolve, not null

Returns:

the resolved date, null if insufficient information to create a date

Throws:

`DateTimeException` - if the date cannot be resolved, typically because of a conflict in the input data

period

```
default ChronoPeriod period(int years,
                            int months,
                            int days)
```

Obtains a period for this chronology based on years, months and days.

This returns a period tied to this chronology using the specified years, months and days. All supplied chronologies use periods based on years, months and days, however the `ChronoPeriod` API allows the period to be represented using other units.

Implementation Requirements:

The default implementation returns an implementation class suitable for most calendar systems. It is based solely on the three units. Normalization, addition and subtraction derive the number of months in a year from the `range(ChronoField)`. If the number of months within a year is fixed, then

the calculation approach for addition, subtraction and normalization is slightly different.

If implementing an unusual calendar system that is not based on years, months and days, or where you want direct control, then the `ChronoPeriod` interface must be directly implemented.

The returned period is immutable and thread-safe.

Parameters:

`years` - the number of years, may be negative

`months` - the number of years, may be negative

`days` - the number of years, may be negative

Returns:

the period in terms of this chronology, not null

epochSecond

```
default long epochSecond(int prolepticYear,
                         int month,
                         int dayOfMonth,
                         int hour,
                         int minute,
                         int second,
                         ZoneOffset zoneOffset)
```

Gets the number of seconds from the epoch of 1970-01-01T00:00:00Z.

The number of seconds is calculated using the proleptic-year, month, day-of-month, hour, minute, second, and zoneOffset.

Parameters:

`prolepticYear` - the chronology proleptic-year

`month` - the chronology month-of-year

`dayOfMonth` - the chronology day-of-month

`hour` - the hour-of-day, from 0 to 23

`minute` - the minute-of-hour, from 0 to 59

`second` - the second-of-minute, from 0 to 59

`zoneOffset` - the zone offset, not null

Returns:

the number of seconds relative to 1970-01-01T00:00:00Z, may be negative

Throws:

DateTimeException - if any of the values are out of range

Since:

9

epochSecond

```
default long epochSecond(Era era,
                         int yearOfEra,
                         int month,
                         int dayOfMonth,
                         int hour,
                         int minute,
                         int second,
                         ZoneOffset zoneOffset)
```

Gets the number of seconds from the epoch of 1970-01-01T00:00:00Z.

The number of seconds is calculated using the era, year-of-era, month, day-of-month, hour, minute, second, and zoneOffset.

Parameters:

`era` - the era of the correct type for the chronology, not null

`yearOfEra` - the chronology year-of-era

`month` - the chronology month-of-year

`dayOfMonth` - the chronology day-of-month

`hour` - the hour-of-day, from 0 to 23

`minute` - the minute-of-hour, from 0 to 59

`second` - the second-of-minute, from 0 to 59

`zoneOffset` - the zone offset, not null

Returns:

the number of seconds relative to 1970-01-01T00:00:00Z, may be negative

Throws:

DateTimeException - if any of the values are out of range

Since:

9

isIsoBased

```
default boolean isIsoBased()
```

Checks if this chronology is ISO based.

An ISO based chronology has the same basic structure as the ISO chronology, i.e., the chronology has the same number of months, the number of days in each month, and day-of-year and leap years are the same as ISO chronology. It also supports the concept of week-based-year of ISO chronology. For example, the Minguo, ThaiThaiBuddhist and Japanese chronologies are ISO based.

Implementation Requirements:

The default implementation returns `false`.

Returns:

`true` only if all the fields of IsoFields are supported by this chronology. Otherwise, returns `false`.

Since:

19

See Also:

IsoChronology, JapaneseChronology, MinguoChronology, ThaiBuddhistChronology, IsoFields

compareTo

```
int compareTo(Chronology other)
```

Compares this chronology to another chronology.

The comparison order first by the chronology ID string, then by any additional information specific to the subclass. It is "consistent with equals", as defined by Comparable.

Specified by:

compareTo in interface Comparable<Chronology>

Parameters:

`other` - the other chronology to compare to, not null

Returns:

the comparator value, that is this ID string compared with the `other`'s ID string unless the ID strings are equal and the chronology distinguishes instances using additional information

equals

```
boolean equals(Object obj)
```

Checks if this chronology is equal to another chronology.

The comparison is based on the entire state of the object.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other chronology

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
int hashCode()
```

A hash code for this chronology.

The hash code should be based on the entire state of the object.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
String toString()
```

Outputs this chronology as a `String`.

The format should include the entire state of the object.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this chronology, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA

94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.